TICKET BOOKING SYSTEM FOR INTERNAL DEPARTMENT EVENT

School of Computing

Bachelor of Technology

in

Computer Science & Engineering

By

K. MAHENDRA NAGA SAI (VTU24866)

10211CS224

Full Stack Application Development

Slot: E3

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Ticket Booking System for Internal Department Event" by K. MAHENDRA NAGA SAI (VTU24866) has been carried out in Winter semester of Academic year 2025–2026 (WS2526).

Signature of Faculty

Dr. Hafizur Rahman
Assistant Professor
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
May, 2026

Signature of Course Coordinator

Dr. Sumithra. M
Assistant Professor (Senior Grade)
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(K. MAHENDRA NAGA SAI)

Date: 06/05/2026

ABSTRACT

The Ticket Booking System for Internal Department Event is a web-based application developed using React JS to enable students and faculty to view event details and book tickets online for department-level events such as technical fests and seminars. In academic institutions, managing ticket bookings for internal events manually can be time-consuming and error-prone. This project provides a single-page React JS application where users can view event information including event name, department, date, time, venue, and ticket price. Users can book tickets by entering their name, email ID, department, and number of tickets required. The system validates all input fields, prevents over-booking beyond available tickets, and displays a booking confirmation summary along with the total amount. The application uses functional components, the useState hook for state management, event handling, and conditional rendering. The available ticket count is updated in real time after each booking.

Keywords: React JS, Ticket Booking, Functional Components, useState Hook, Event Handling, Conditional Rendering, Form Validation, Component-Based Architecture.

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CLI	Command Line Interface
CSS	Cascading Style Sheets
DOM	Document Object Model
HTML	Hyper Text Markup Language
JS	JavaScript
JSX	JavaScript XML
npm	Node Package Manager
SPA	Single Page Application
UI	User Interface
UX	User Experience

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

In educational institutions, internal department events such as technical fests, seminars, workshops, and cultural programs are organized regularly. Managing ticket bookings for these events manually involves maintaining physical registers, collecting forms, and tracking available seats, which is time-consuming and prone to errors. Students and faculty often face difficulties in finding event information and completing the booking process efficiently.

The Ticket Booking System for Internal Department Event is developed to address this problem. It is a React JS based single-page web application that allows users such as students and faculty to view complete event details and book tickets online without any manual paperwork. The application displays event information clearly, collects user details through a validated form, prevents overbooking, and

shows a detailed booking confirmation summary after each successful booking.

The system is entirely frontend-based and runs directly in a browser. It uses React JS functional components, the useState hook for managing application state, event handling for form interaction, and conditional rendering to switch between the booking form and the confirmation summary. The project demonstrates the practical use of React concepts in building a real-world web application.

1.2 Aim of the Project

The main aim of this project is to design and develop a React JS based Ticket Booking System for an internal department event. The system allows users to view event details and book tickets online with proper validation and confirmation.

Key objectives include:

- To display event details such as name, department, date, time, venue, and ticket price.
- To allow users to book tickets by entering name, email ID, department, and number of tickets.
- To validate all input fields and display appropriate error messages for invalid inputs.

- To prevent overbooking by checking available ticket count before confirming a booking.
- To display a booking confirmation summary with user name, event name, tickets booked, and total amount.
- To update the available ticket count in real time after each successful booking.
- To implement the application using functional components, useState hook, event handling, and conditional rendering.

1.3 Scope of the Project

The scope of this project covers all features required for online ticket booking for an internal department event. The system is useful for students, faculty, and event coordinators.

- **For Students and Faculty:** Users can view event details and book tickets by filling in the booking form. The system validates the inputs and confirms the booking with a summary.
- **For Event Coordinators:** The system provides real-time ticket availability tracking. The available ticket count is reduced automatically after each booking.

- **For Validation:** All input fields are mandatory. Email ID must be in valid format. Number of tickets must be a positive integer and must not exceed available tickets.
- **For Confirmation:** After successful booking, a summary is displayed showing user name, event name, number of tickets, and total amount.

1.4 Framework

This project is developed using React JS, which is a JavaScript library for building user interfaces. The application is structured using the following components:

- **Frontend (User Interface):** The entire application is built using React JS with JSX syntax and CSS styling. It runs in a browser as a Single Page Application (SPA).
- **State Management:** The useState hook is used to manage form data, error messages, available ticket count, and booking confirmation state.
- **Component Structure:** The application is divided into separate functional components: `EventDetails`, `BookingForm`, and `BookingSummary`, all composed inside the root `App` component.

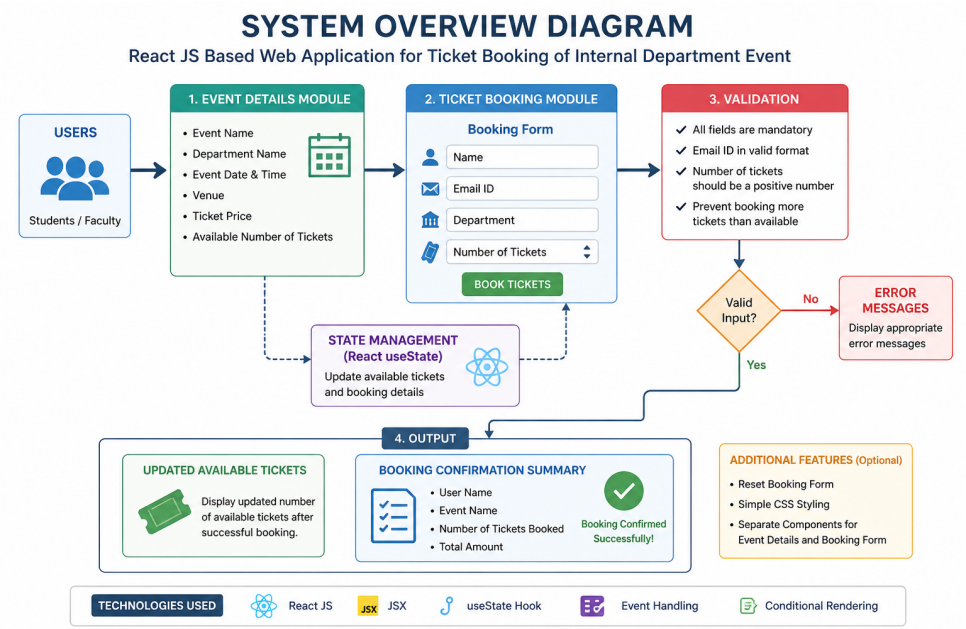

Figure 1.1: System Overview of Ticket Booking System

Figure 1.1 shows the system architecture of the React-based Ticket Booking System. It includes `EventDetails` for displaying event info, `BookingForm` for user input and validation, and `BookingSummary` for confirmation. The `App` component manages state using `useState`, updates ticket availability, and handles component switching through conditional rendering.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

1. Ticket booking systems are widely used in institutions and organizations.
2. Studying existing platforms helps identify their features and limitations.
3. **Google Forms:** Simple and easy to use for event registration, but it lacks structured event display and does not support real-time ticket tracking.
4. **Eventbrite:** Provides advanced features such as ticket management and analytics, but it is complex for small internal or academic events.
5. **BookMyShow:** Supports online payments and seat selection, but it is not suitable for academic or internal department events.

6. **College ERP Systems:** Offer integrated event management modules, but they are large, complex, and not lightweight.

7. **Proposed System:** A simple React JS-based application designed for internal department events, focusing on event display, validation, ticket tracking, and booking confirmation without backend dependency.

8. Most existing systems require user login or account creation, which can make the booking process time-consuming for quick event registrations.

9. Many platforms depend on continuous internet connectivity and server-side processing, making them less efficient in low-resource or offline scenarios.

10. Existing solutions often include unnecessary features for small-scale events, increasing complexity instead of providing a focused and user-friendly experience.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The application is developed using React JS. Node.js and npm are required to set up the development environment. The project is created using the Create React App tool, which provides a pre-configured development environment with a local development server, hot reloading, and build tools. The application runs in a web browser on localhost.

The tools and technologies used in this project include:

- **React JS** – JavaScript library for building user interfaces
- **JSX** – JavaScript XML syntax used to write component templates
- **CSS** – Used for styling the components
- **Node.js and npm** – Required for running the development server and managing packages

- **Create React App** – Tool for bootstrapping the React project

3.1.2 Structure of the Project

The project follows a component-based architecture. Each major UI section is implemented as a separate functional component.

Table 3.1: Project Folder Structure

Source Folder (src)
App.js
App.css
index.js
components/EventDetails.js
components/BookingForm.js
components/BookingSummary.js
Public Folder (public)
index.html

Table 3.1 has each component has a specific purpose. `EventDetails` displays the event information. `BookingForm` collects and validates user input. `BookingSummary` displays the booking confirmation. `App.js` is the root component that composes all other components and manages shared state.

3.1.3 Execution Procedure

To run the application, the following commands are used in the terminal:

```
npx create-react-app ticket-booking
```

```
cd ticket-booking
npm start
```

The application opens automatically in the browser at:

```
http://localhost:3000/
```

Users can view the event details, fill in the booking form, and receive a booking confirmation.

3.2 Component Implementation

3.2.1 App Component

The App component is the root component. It holds the shared state for available ticket count and the booking confirmation data. It renders the `EventDetails`, `BookingForm`, and `BookingSummary` components conditionally.

Table 3.2: App Component – State and Structure

```
const [available, setAvailable] = useState(EVENT.totalTickets);
const [booking, setBooking] = useState(null);

const handleBook = (data) => {
  setAvailable(a => a - data.tickets);
  setBooking(data);
};
```

Table 3.2 presents the state management and structural design of the App component. It highlights how data is organized and managed to

enable efficient interaction between different parts of the application.

3.2.2 EventDetails Component

The `EventDetails` component displays the event information. It receives the current available ticket count as a prop and shows a live availability progress bar. The event data includes event name, department, date, time, venue, and ticket price.

Table 3.3: EventDetails Component – Props and Display

```
function EventDetails({ available }) {
  const sold = EVENT.totalTickets - available;
  const pct = Math.round((sold / EVENT.totalTickets) * 100);
  return (
    <div>
      <h2>{EVENT.name}</h2>
      <p>Department: {EVENT.department}</p>
      <p>Date: {EVENT.date} | Time: {EVENT.time}</p>
      <p>Venue: {EVENT.venue}</p>
      <p>Price: Rs.{EVENT.ticketPrice} per ticket</p>
      <p>Available: {available} of {EVENT.totalTickets}</p>
    </div>
  );
}
```

Table 3.3 describes the props and display logic of the EventDetails component. It illustrates how event-related data is passed and rendered within the user interface.

3.2.3 BookingForm Component

The `BookingForm` component collects user inputs: name, email ID, department, and number of tickets. It validates all fields before

submitting. Validation checks include mandatory fields, email format, positive ticket count, and overbooking prevention.

Table 3.4: BookingForm Component – Validation Logic

```
const validate = () => {
  const e = {};
  if (!form.name.trim()) e.name = "Name is required.";
  if (!form.email.trim()) e.email = "Email is required.";
  else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(form.email))
    e.email = "Enter a valid email address.";
  if (!form.department.trim())
    e.department = "Department is required.";
  if (!form.tickets) e.tickets = "Number of tickets required.";
  else if (Number(form.tickets) <= 0)
    e.tickets = "Enter a positive number.";
  else if (Number(form.tickets) > available)
    e.tickets = "Only " + available + " ticket(s) available.";
  return e;
};
```

Table 3.4 outlines the validation logic implemented in the BookingForm component. It demonstrates how user inputs are verified to ensure accuracy, completeness, and proper booking flow.

3.2.4 BookingSummary Component

The `BookingSummary` component is displayed after a successful booking. It uses conditional rendering inside the `App` component. It shows the user name, event name, number of tickets booked, and total amount. A reset button allows the user to book another ticket.

Table 3.5: BookingSummary Component – Confirmation Display

```
function BookingSummary({ booking, onBookAnother }) {
  const total = booking.tickets * EVENT.ticketPrice;
  return (
    <div>
      <h2>Booking Confirmed!</h2>
      <p>Name: {booking.name}</p>
      <p>Event: {EVENT.name}</p>
      <p>Tickets: {booking.tickets}</p>
      <p>Total Amount: Rs.{total}</p>
      <button onClick={onBookAnother}>
        Book Another Ticket
      </button>
    </div>
  );
}
```

Table 3.5 presents the confirmation display handled by the BookingSummary component. It shows how booking details are summarized and presented to the user after successful submission.

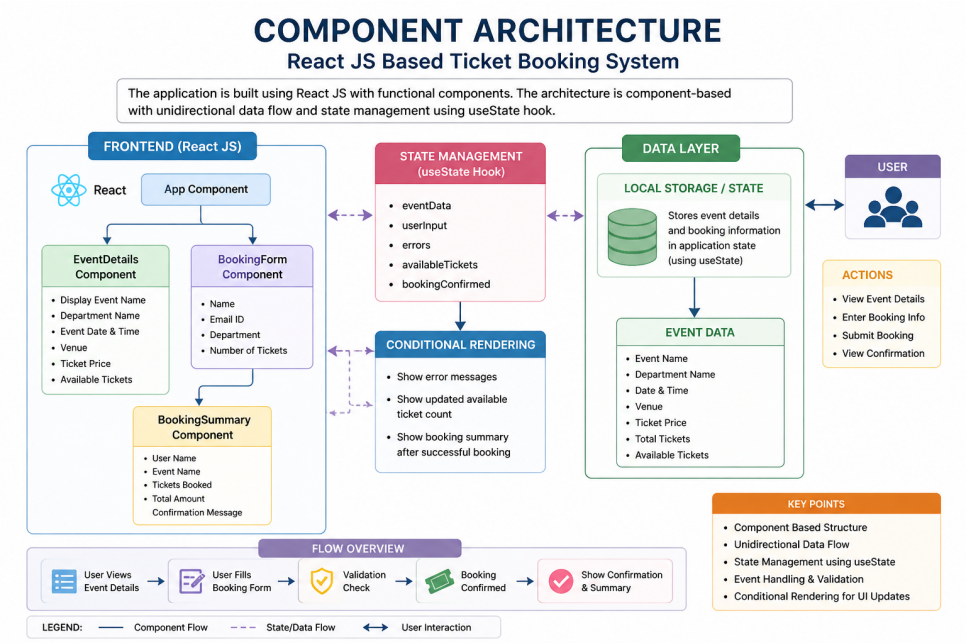

Figure 3.1: Component Architecture of Ticket Booking System

Figure 3.1 illustrates the component architecture of the Ticket Booking System developed using React JS. The application follows a modular and component-based design, where the user interface is divided into reusable functional components.

At the core of the architecture is the `App` component, which acts as the main controller of the application. It manages the overall state, including the available ticket count and booking data, using the `useState` hook. The `App` component coordinates the interaction between all other components through props and callback functions.

The `EventDetails` component is responsible for displaying the event information such as event name, department, date, time, venue, ticket price, and available tickets. It receives data from the `App` component as props and updates dynamically based on state changes.

The `BookingForm` component handles user input and form validation. It collects details such as name, email ID, department, and number of tickets. It ensures that all inputs are valid and prevents overbooking by checking the available ticket count before submission.

The `BookingSummary` component is displayed after a successful booking. It presents a confirmation message along with booking details such as user name, number of tickets booked, and total amount. This component is rendered conditionally based on the application state.

The interaction between these components is managed through props and event handling, ensuring smooth data flow and real-time updates. This architecture demonstrates the effective use of React JS concepts such as functional components, state management, and conditional rendering.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a component-based architecture using React JS. The application is divided into reusable functional components. Each component is responsible for a specific part of the user interface. State is managed using the useState hook and is passed between components using props and callback functions.

The overall architecture flow is:

```
User Browser
    |
React Application (index.js)
    |
App Component (App.js)
    |
    |--- EventDetails Component
```

```
|
|--- BookingForm Component (form visible)
OR
|--- BookingSummary Component (after booking)
```

The App component acts as the state manager. It holds the available ticket count and booking data. It passes these values as props to child components and receives updated values through callback functions such as handleBook and handleBookAnother.

4.2 Data Flow Diagram

The data flow of the Ticket Booking System describes how information moves between components and state.

- Event data is stored as a constant in App.
- EventDetails displays available tickets via props.
- BookingForm handles user input.
- On submit, input is validated and onBook is called.
- App updates ticket count and booking state.
- Conditional rendering switches to BookingSummary.
- BookingSummary shows confirmation details.

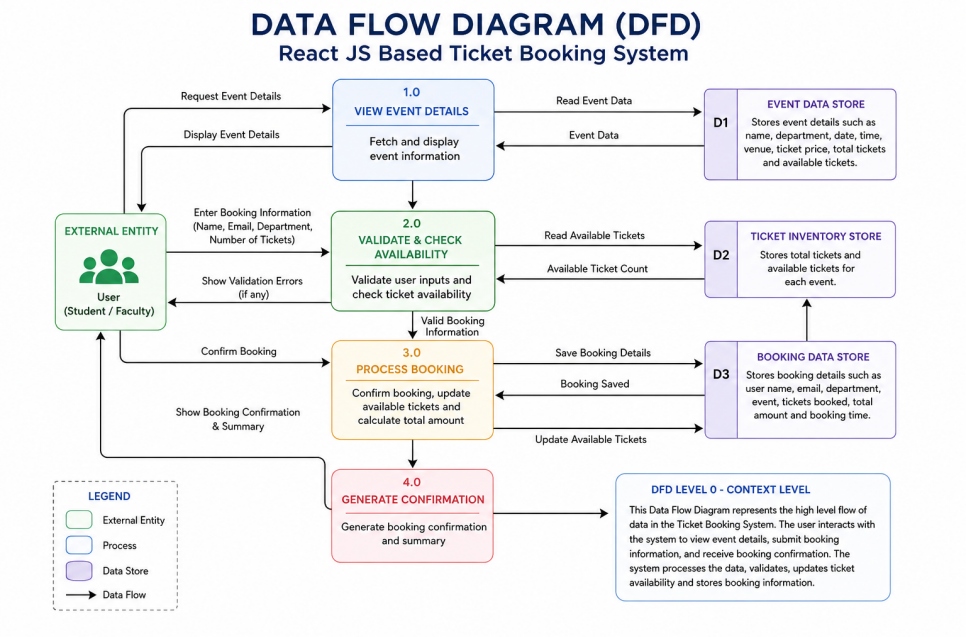

Figure 4.1: Data Flow Diagram of Ticket Booking System

Figure 4.1 illustrates the Data Flow Diagram (DFD) of the Ticket Booking System for an internal department event. The diagram represents how data moves between the user, system components, and internal processing stages.

The process begins when the user accesses the application through a web browser and views the event details. The event information, including event name, department, date, time, venue, ticket price, and available tickets, is displayed by the system.

The user then enters booking details such as name, email ID, department, and number of tickets through the booking form. This input data is sent to the validation process, where the system checks for mandatory fields, correct email format, and valid ticket count. The system also verifies ticket availability to prevent overbooking.

If validation fails, appropriate error messages are returned to the user, prompting correction of inputs. If validation is successful, the booking data is processed, and the available ticket count is updated accordingly in the system state.

Finally, the system generates a booking confirmation, which is displayed to the user through the booking summary module. This includes details such as user name, number of tickets booked, and total amount payable.

The DFD clearly demonstrates the flow of data between input, processing, and output stages, ensuring that the system operates efficiently with proper validation and real-time updates.

4.3 Module 1: Event Details Module

The Event Details Module is implemented in the `EventDetails` functional component. It displays the following information about the event:

- Event Name
- Department Name
- Event Date and Time
- Venue

- Ticket Price
- Available Number of Tickets

The available ticket count is passed as a prop from the `App` component. The module also displays a visual progress bar that shows how many tickets have been sold. When fewer than 20 tickets remain, a low-availability warning is displayed. When tickets are fully sold out, a Sold Out indicator is shown and the booking form is disabled.

This module uses functional component structure, JSX for rendering, and conditional rendering for the warning and sold-out messages.

4.4 Module 2: Ticket Booking Module

The Ticket Booking Module is implemented in the `BookingForm` functional component. It allows users to enter the following details:

- Full Name
- Email ID
- Department
- Number of Tickets Required

The module uses the useState hook to manage form field values and error messages separately. When the user clicks the Confirm Booking

button, the validate function checks all fields. If errors exist, they are displayed below the respective fields. If validation passes, the `onBook` callback is called with the form data.

The module also displays a real-time total price preview below the ticket count field when a valid number is entered. A Reset button clears all fields and error messages.

4.5 Module 3: Booking Confirmation Module

The Booking Confirmation Module is implemented in the `BookingSummary` functional component. It is displayed conditionally after a successful booking using React's conditional rendering pattern inside the `App` component.

The confirmation summary displays:

- User Name
- Email Address
- Department
- Event Name
- Event Date and Time
- Venue
- Number of Tickets Booked

- Total Amount Payable

A Book Another Ticket button calls the `onBookAnother` callback, which resets the booking state in the `App` component and returns to the booking form.

Advantages of this Project

a) Component-Based Design: The application is structured into separate functional components, making the code modular, readable, and easy to maintain or extend.

b) Real-Time State Management: The useState hook manages the available ticket count and booking data in real time. The UI updates instantly after every booking without page reload.

c) Input Validation: All fields are validated before submission. Email format, positive ticket count, and overbooking checks prevent invalid data from being processed.

d) Overbooking Prevention: The system checks available tickets before confirming a booking. Users cannot book more tickets than what is available.

e) Booking Confirmation: A detailed booking summary is displayed after successful booking, giving users clear confirmation of their reservation.

f) No Backend Required: The entire application runs in the browser.

No server, database, or API setup is needed, making it lightweight and easy to run.

g) Reusable Components: Each component is self-contained and reusable. The `EventDetails` and `BookingForm` components can be easily adapted for different events.

h) Reset Functionality: Users can reset the booking form after successful booking to book tickets again, improving usability.

i) User-Friendly Error Messages: Clear and specific error messages are shown for each invalid field, helping users correct their inputs quickly.

j) Modern React Concepts: The application demonstrates the use of functional components, JSX, useState hook, props, event handling, and conditional rendering as required by the project specification.

Disadvantages of this Project

- **No Persistent Storage:** Booking data is stored only in component state. When the page is refreshed, all booking information is lost.
- **No Backend Integration:** The system does not connect to any server or database. It cannot store or retrieve booking records permanently.
- **Single Event Support:** The current implementation supports only one event at a time. Multiple events are not supported without

modifying the code.

- **No Payment Integration:** The system does not process payments. It only calculates the total amount without a payment gateway.
- **No User Authentication:** There is no login system for users. Anyone can book tickets without identity verification.
- **No Email Notification:** After booking, no confirmation email is sent to the user since there is no backend email service.
- **No Duplicate Booking Check:** The system does not prevent the same user from booking tickets multiple times using the same email address.
- **Basic Styling:** The user interface uses simple CSS styling and may not be fully responsive on all screen sizes.
- **No Admin Panel:** There is no admin interface to manage events, view bookings, or update ticket availability.
- **No Search or Filter:** The system does not support searching or filtering events by category, department, or date.

Chapter 5

RESULTS AND DISCUSSIONS

After developing the Ticket Booking System for Internal Department Event using React JS, the application worked successfully. The home page displays a two-column layout with event details on the left and the booking form on the right. The event details panel shows the event name, department, date, time, venue, ticket price, and a live availability progress bar.

The application was tested with the event "Tantraz – Technical Fest 2026" from the Department of Computer Science and Engineering. The admin-defined event had 100 total tickets priced at Rs.150 each. Users were able to fill in the booking form, and the system correctly validated all inputs and displayed error messages for invalid data. After successful booking, the confirmation summary was displayed with all booking details and the total amount. The available ticket count was updated correctly after each booking.

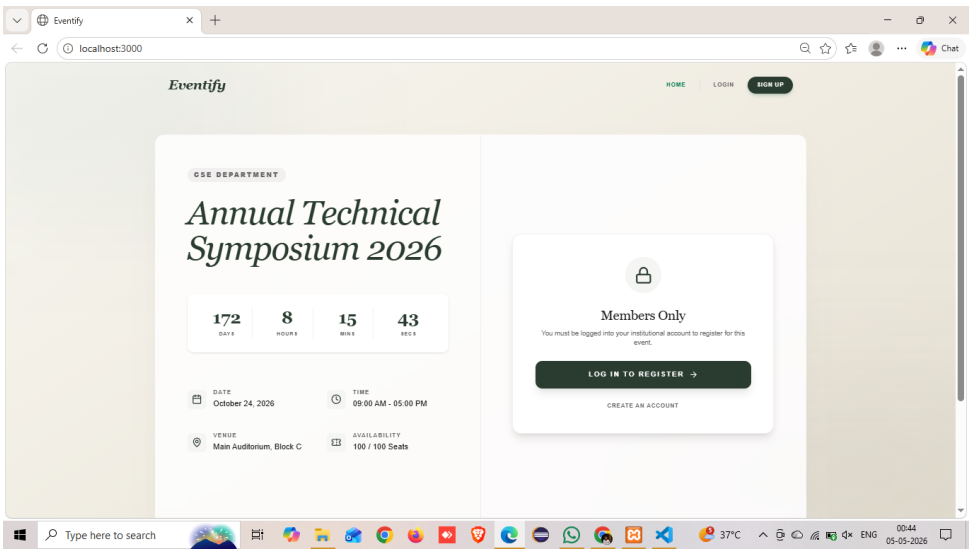

Figure 5.1: Home Page – Event Details and Booking Form

Figure 5.1 shows the home page of the Ticket Booking System. The interface is divided into two main sections: the event details panel and the booking form. The event details section displays information such as event name, department, date, time, venue, ticket price, and available ticket count. The booking form allows users to enter their details, including name, email ID, department, and number of tickets. The layout is designed to provide a clear and user-friendly interface, enabling users to view event information and perform booking operations on a single page.

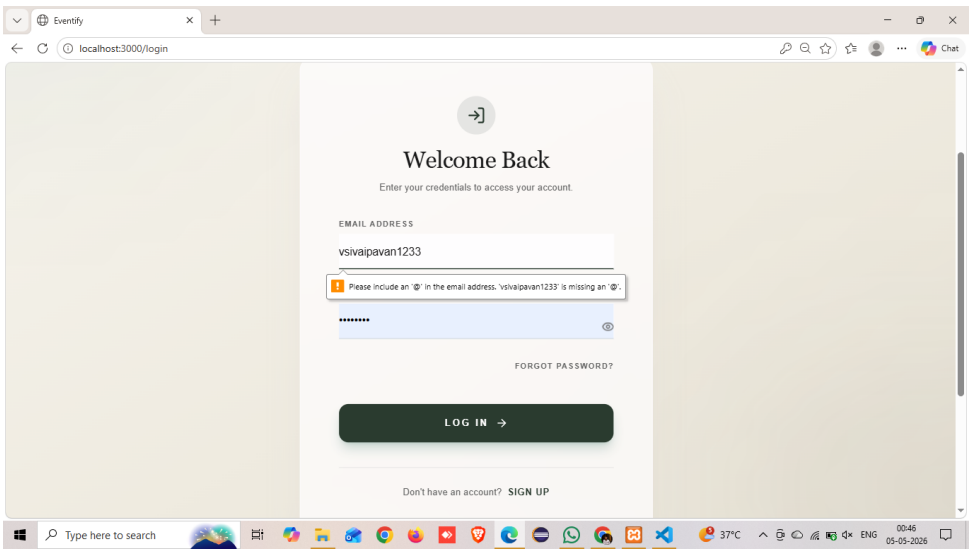

Figure 5.2: Booking Form with Validation

Figure 5.2 illustrates the booking form with validation. The system validates all user inputs before processing the booking request. Mandatory fields such as name, email ID, department, and ticket count are checked for correctness. If invalid data is entered, appropriate error messages are displayed to guide the user. The system also ensures that the number of tickets entered is positive and does not exceed the available ticket count, thereby preventing overbooking.

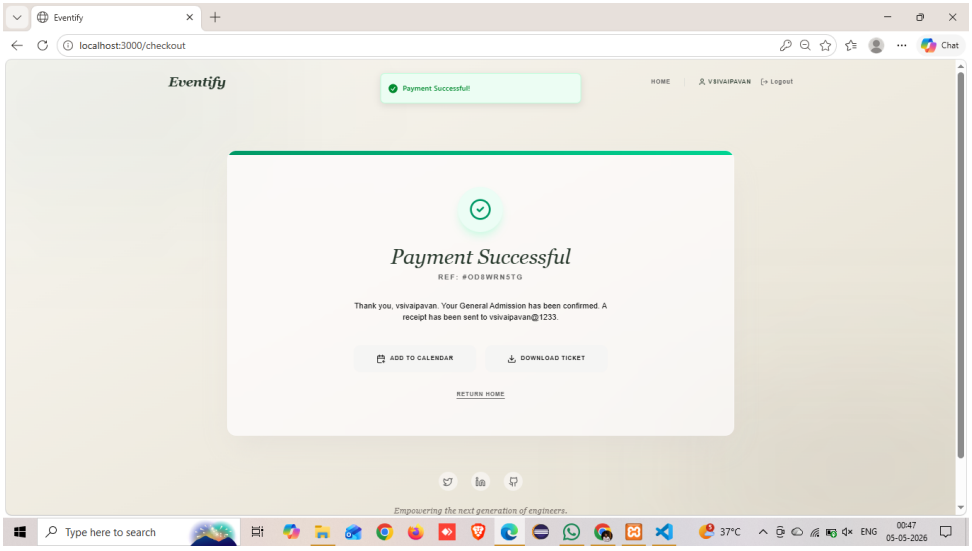

Figure 5.3: Booking Confirmation Summary

Figure 5.3 shows the booking confirmation summary displayed after a successful ticket booking. The summary includes details such as user name, event name, number of tickets booked, and total amount payable. This confirmation screen provides clear feedback to the user, ensuring that the booking process has been completed successfully. It also includes an option to book another ticket, improving usability and user interaction.

Table 5.1: Module Result

Module / Feature	**Result**
Event Details Display	Working
Ticket Availability Bar	Working
Mandatory Field Validation	Working
Email Format Validation	Working
Positive Ticket Count Check	Working
Overbooking Prevention	Working
Real-Time Total Price	Working
Booking Confirmation Summary	Working
Available Count Update	Working
Reset Form Functionality	Working

Table 5.1 presents the results of various modules implemented in the Ticket Booking System. Each module has been tested to ensure proper functionality and performance.

All core features, including event details display, ticket availability tracking, input validation, and booking confirmation, are functioning correctly. The system successfully validates user inputs such as email format and ticket count, and prevents overbooking by checking ticket availability in real time.

Additionally, features such as real-time total price calculation, booking confirmation summary, and reset functionality have been implemented effectively. The results indicate that all modules are working as expected, demonstrating the correctness and reliability of the system.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

In conclusion, the Ticket Booking System for Internal Department Event is a well-structured and functional React JS application that successfully addresses the challenges of manual ticket booking for academic events. Traditional methods of collecting registrations through paper forms or spreadsheets are time-consuming, error-prone, and difficult to manage. This project provides a simple and efficient digital solution to replace manual processes.

The application provides a clean and organized interface where users can view complete event details and book tickets by filling in a validated form. The form validates all input fields, including email format, ticket count, and overbooking prevention, ensuring that only valid bookings are accepted. After a successful booking, a detailed confirmation

summary is displayed showing the user name, event name, number of tickets, and total amount. The available ticket count is updated in real time after each booking.

The project is developed using React JS functional components, the useState hook, event handling, conditional rendering, and JSX. These React concepts are properly demonstrated through the modular component structure of `App`, `EventDetails`, `BookingForm`, and `BookingSummary`. The application runs entirely in the browser without any backend requirement, making it lightweight and easy to demonstrate.

All features specified in the project requirements are implemented and working successfully. The project demonstrates how a practical ticket booking problem can be solved using React JS and modern frontend development techniques.

6.2 Future Enhancements

In the future, the system can be improved by adding more advanced features.

Backend Integration: A Node.js or Spring Boot backend can be connected to store booking records permanently in a database.

Email Notification: An email confirmation can be sent to the user

after successful booking using a backend email service.

Multiple Events: The system can be extended to display and manage multiple events simultaneously.

User Authentication: A login system can be added for students and faculty to track their booking history.

Payment Gateway: Online payment integration can be added to support paid event tickets.

Admin Dashboard: An admin panel can be developed to manage events, view all bookings, and export reports.

QR Code Generation: A QR code can be generated for each confirmed booking and used for entry verification at the event.

Responsive Design: The user interface can be improved to be fully responsive across mobile, tablet, and desktop screens.

Duplicate Booking Prevention: Email-based duplicate booking checks can be implemented to prevent the same user from booking multiple times.

Cloud Deployment: The application can be deployed on platforms such as Vercel or Netlify so that users can access it from anywhere.

Overall, these enhancements can make the Ticket Booking System more powerful, scalable, and ready for real-time use in academic institutions.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

This chapter describes how the Ticket Booking System for Internal Department Event addresses Complex Engineering Problems (CEP), Complex Engineering Activities (CEA), and Sustainable Development Goals (SDGs). The project integrates React JS concepts such as functional components, useState hook, event handling, conditional rendering, and JSX to build a complete frontend web application.

The system replaces manual ticket booking with a digital browser-based platform. Students and faculty can view event details, book tickets online, and receive a confirmation summary instantly. From an engineering perspective, the project involves application design, component architecture, state management, form validation, and user interface development.

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

Level	**Attribute**	**Characteristics**	**WKs**	**Justification**
WP1	Depth of Knowledge	In-depth engineering knowledge	WK3, WK4, WK5, WK6	Requires React JS, component-based design, state management with useState, JSX, event handling, and CSS styling knowledge.
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances simple user experience with strict input validation, overbooking prevention, and real-time state updates.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	Involves component decomposition, prop flow design, state management strategy, and validation logic planning.
WP4	Familiarity	Partially new problems	WK4, WK8	Converts manual campus ticket booking into a digital React JS SPA with real-time availability tracking and confirmation.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom validation logic, ticket count management, conditional rendering flow, and booking summary display.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Supports students, faculty, and event coordinators by simplifying event discovery and ticket booking.
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Integrates multiple React components with shared state, props, callbacks, and conditional rendering into a unified SPA.

Table 7.1 shows how the proposed system addresses Complex Engineering Problems (CEP) using modern technologies, validation mechanisms, and efficient system design.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	**Attribute**	**Characteristics**	**Justification based on the Project**
EA1	Range of Resources	Use of diverse technologies	Utilizes React JS, JSX, useState hook, CSS, and JavaScript for building a complete single-page ticket booking application.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between EventDetails, BookingForm, and BookingSummary components through shared state and callback props.
EA3	Innovation	Application of modern technologies	Implements a browser-based ticket booking SPA using React functional components with real-time state management and conditional rendering.
EA4	Consequences to Society and Environment	Impact on users and society	Improves access to event information, reduces paper usage, and simplifies the ticket booking process for students and faculty.
EA5	Familiarity	Beyond standard practices	Combines component-based architecture, state management, form validation, and dynamic UI rendering into a unified application.

Table 7.2 presents the mapping of the developed Ticket Booking System to Complex Engineering Activities (CEA). The project demonstrates the application of modern web technologies, effective component interaction, and innovative design practices to address real-world problems while considering societal and environmental impacts.

7.3 SDG Mapping

Table 7.3: SDG Mapping for the Project

SDG No.	**SDG Title**	**Relevance to the Project**
SDG 4	Quality Education	Facilitates student and faculty participation in academic events, workshops, and technical fests by simplifying ticket booking.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a browser-based ticket booking application built with modern React JS.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitizing the event ticket booking process and reducing manual administrative work.
SDG 12	Responsible Consumption and Production	Eliminates paper-based registration forms by enabling online ticket booking, thereby reducing paper waste in campus event management.
SDG 16	Peace, Justice and Strong Institutions	Ensures transparent and fair ticket booking with validation checks and overbooking prevention for equitable access to events.

Table 7.3 illustrates how the developed Ticket Booking System aligns with selected United Nations Sustainable Development Goals (SDGs). The project contributes to improving digital accessibility, promoting sustainability, and enhancing transparency within educational institutions.

REFERENCES

1. React JS Documentation. “React – A JavaScript library for building user interfaces.” Available at: https://reactjs.org/docs/getting-started.html

2. React Hooks Documentation. “Introducing Hooks.” Available at: https://reactjs.org/docs/hooks-intro.html

3. React useState Hook. “Using the State Hook.” Available at: https://reactjs.org/docs/hooks-state.html

4. Create React App Documentation. “Getting Started with Create React App.” Available at: https://create-react-app.dev/docs/getting-started/

5. MDN Web Docs. “JavaScript Reference Guide.” Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript

6. MDN Web Docs. “HTML and CSS Documentation.” Available at: https://developer.mozilla.org/en-US/docs/Web

7. W3Schools. “React Tutorial.” Available at: https://www.w3schools.com/react/

8. Baeldung. “Introduction to React.” Available at: https://www.baeldung.com/react

9. Node.js Documentation. “Node.js Official Documentation.” Available at: https://nodejs.org/en/docs/

10. npm Documentation. “npm – Node Package Manager.” Available at: https://docs.npmjs.com/

11. JSX in Depth. “JSX Syntax and Usage.” Available at: https://reactjs.org/docs/jsx-in-depth.html

12. Event Management Concepts. “Online Event Registration Systems Overview.” Available at: https://www.tutorialspoint.com/event-management